

SQLite Data Types

 <https://github.com/learn-co-curriculum/python-p3-sqlite-data-types>  <https://github.com/learn-co-curriculum/python-p3-sqlite-data-types/issues/new>

Learning Goals

- Use SQL to store data and retrieve it later on.
- Use SQLite to build relational databases on your computer.

Key Vocab

- **SQL (Structured Query Language)**: a programming language that is used to manage relational databases and perform operations on the data that they contain.
- **Relational Database**: a collection of data that is organized in well-defined relationships. The most common type of database.
- **Query**: a statement used to return data from a database.
- **Table**: a collection of related data in a database. Composed of rows and columns. Similar to a class in Python.
- **Row**: a single record in a database table. Each column represents an attribute of the record. Similar to an object in Python.
- **Column**: a single field in a database table. Each row contains values in each column. Similar to a Python object's attributes.
- **Schema**: a blueprint of the construction of the tables in a database and how they relate to one another.

Why Do Data Types Matter?

We've learned that when we create a table, we need to include a name for it as well as define at least one column. We define columns in a **CREATE** statement by including a name and a **datatype** to let SQLite know the *kind* of data we will be storing there. The practice of explicitly declaring a type is known as "typing."

Why is it important that we use typing in our database? Simply put, typing allows us to exercise some level of control over our data. Typing not only informs our database of the kind of data we plan to store in a column, but it also restricts it. For instance, look at the `age` column below in our `cats` table. What do we mean by age? What if we had this:

name	breed	age
Maru	Scottish Fold	3
Hannah	Tabby	two
Lil' Bub	American Shorthair	5.5

Did we intend `age` to be represented as a whole number, a word, or a decimal? If we asked you to add up the ages of all the cats you could simply convert the 'two' to 2 in your head, but your database can't do that. It doesn't have that ability, because the logic involved in converting a word into a number would be dense and inefficient. What about different languages? What about different spellings? Capitalization, typos, or different hyphenation conventions? These are just some reasons this might start to get crazy. In other words, because databases are designed to store large amounts of data, they are very concerned with storing, accessing, and acting upon that data as efficiently and normally as possible.

Typing gives us the ability to perform all kinds of operations with predictable results. For instance, the ability to perform math operations like `SUM` — i.e. summing integers — doesn't just depend on everything being an integer of some sort but would also expect it. If you tried, for example, to `SUM` all of the cats in the above table, SQLite would actually attempt to convert, or cast, their type to something it can `SUM`. It would try to convert anything it can to an `INTEGER` and ignore alpha characters. This can lead to real problems. Without typing, our data might get complicated and messy, and it would be difficult to ask the database questions about large sets of data.

We're going to adhere strictly to only storing data that fits with the datatype we have given to a particular column.

Datatypes

Different database systems also have different datatypes available, which are important and useful to know whenever you are dealing with those systems. SQLite is a good starting point to learn about datatypes because it only has five basic categories of datatypes; they are:

- `NULL`

- **TEXT**
- **INTEGER**
- **REAL**
- **BLOB**

Let's explore each category in more detail.

NULL

In a database, **NULL** represents "no value", like `null` in JavaScript or `None` in Python.

TEXT

Any alphanumeric characters which we want to represent as plain text. The body of this paragraph is text. Your name is text. Your email address is a piece of text. Your height, weight, and age, however, are probably not.

INTEGER

Anything we want to represent as a whole number. If it's a number and contains no letter or special characters or decimal points then we should store it as an integer. If we use it to perform math or create a comparison between two different rows in our database, then we definitely want to store it as an integer. If it's just a number, it's generally not a bad idea to store it as an integer. You might never add two house address numbers together, but you might want to sort them numerically. For example, in the preceding case you might want to get the biggest number and not the longest piece of text.

REAL

Anything that's a plain old decimal like 1.3 or 2.25. SQLite will store decimals up to 15 characters long. You can store 1.2345678912345 or 1234.5678912345, but 1.23456789123456789 would only store 1.2345678912345. In other database systems this is called 'double precision.'

With these three types in hand, we are going to be able to work our way through the next several topics, and this whole typing concept is going to quickly become second nature for you.

BLOB

You may encounter the **BLOB** datatype while you're Googling or doing any further reading on SQLite. For now, we will not use **BLOB**. It is generally used for holding binary data.

Bonus: Note on SQLite

To increase its compatibility with other database engines (e.g. MySQL or PostgreSQL), SQLite allows the programmer to use other common datatypes outside of the four mentioned above. We can refer to **TEXT INTEGER REAL BLOB** as datatype "categories". All other common datatypes are lumped into one of the four existing datatypes recognized by SQLite.

For example, **INT** is a common datatype used outside of SQLite. SQLite won't complain if you define a column as an **INT** datatype. It will simply lump it into the **INTEGER** category and store it as such.

Boolean values are also stored as integers (0 for false, 1 for true).

To accommodate this, SQLite has a pretty complicated system of categorizing datatypes that involves **Storage Classes**, **Type Affinities**, and **Datatypes**. For a deeper dive, check out the [SQLite3 Documentation on Datatypes](http://www.sqlite.org/datatype3.html)  (<http://www.sqlite.org/datatype3.html>)

Conclusion

Unlike dynamic languages like Python and JavaScript, SQL enforces strictness when it comes to datatypes. While it does restrict our ability to toss any old data we want into a particular column of a database, these restrictions work to our favor by enforcing consistency and predictability.

Resources

- [SQL Tutorial - W3Schools](https://www.w3schools.com/sql/)  (<https://www.w3schools.com/sql/>)
- [Documentation - SQLite](https://www.sqlite.org/docs.html)  (<https://www.sqlite.org/docs.html>)

- [SQLite - VisualStudio Marketplace](https://marketplace.visualstudio.com/items?itemName=alexcvzz.vscode-sqlite) ➞ [_ \(https://marketplace.visualstudio.com/items?itemName=alexcvzz.vscode-sqlite\)](https://marketplace.visualstudio.com/items?itemName=alexcvzz.vscode-sqlite)
- [Datatypes in SQLite - SQLite](http://www.sqlite.org/datatype3.html) ➞ [_ \(http://www.sqlite.org/datatype3.html\)](http://www.sqlite.org/datatype3.html)
- [SQLite Keywords - SQLite](https://www.sqlite.org/lang_keywords.html) ➞ [_ \(https://www.sqlite.org/lang_keywords.html\)](https://www.sqlite.org/lang_keywords.html)
- [ZetCode sqlite3 Tutorial](http://zetcode.com/db/sqlite/) ➞ [_ \(http://zetcode.com/db/sqlite/\)](http://zetcode.com/db/sqlite/)